Introducción a la Ingeniería de Datos

Maestría en Inteligencia Artificial - MIA 03

Facultad Politécnica - Universidad Nacional de Asunción

Clase 5: Transformación de Datos con dbt

Contenidos

Contexto: La T en ELT 3
¿Qué es dbt? 6
Instalación y Configuración 12
Modelos en dbt 23
Materializations 34
Ejecutar dbt 40
Actividad Asincrónica 46

Contexto: La T en ELT

Recordatorio: Arquitectura del Curso

APIs / DBs / Archivos

↓

Airbyte - Extract & Load ✓

↓

MotherDuck - Storage ✓

↓

dbt - Transform ← HOY

↓

Metabase - Visualize

Objetivos de la Clase

Al finalizar esta clase podrás

1. Entender qué es **dbt** y por qué es revolucionario
2. Instalar y configurar **dbt-duckdb** con MotherDuck
3. Crear un **proyecto dbt** desde cero
4. Escribir **modelos** en SQL (staging, intermediate, marts)
5. Entender **materializations** (view, table, incremental)
6. Usar **refs** para crear dependencias entre modelos
7. Ejecutar y debuggear tu pipeline de transformación

¿Qué es dbt?

dbt: data build tool

Definición

dbt es una herramienta que permite transformar datos en el warehouse escribiendo SQL modular, con testing, documentación y control de versiones integrados.

dbt: data build tool

Lo que dbt hace

- Transformar datos (la T de ELT)
- Testing de datos
- Documentación automática
- Linaje de datos
- Manejo de dependencias

Lo que dbt NO hace

- Extraer datos
- Cargar datos
- Orquestar pipelines
- Visualizar datos
- Almacenar datos

¿Por Qué dbt Revolucionó la Industria?

Antes de dbt	Con dbt
SQL en stored procedures	SQL en archivos versionados (Git)
Sin testing de datos	Tests integrados y extensibles
Documentación manual	Documentación generada automáticamente
Dependencias implícitas	Dependencias explícitas con DAG
Difícil de debuggear	SQL compilado visible
Sin estándares	Convenciones y mejores prácticas

dbt trajo prácticas de ingeniería de software al mundo de datos

El Rol del Analytics Engineer

Un nuevo rol emergente

dbt popularizó el rol de **Analytics Engineer** - el puente entre Data Engineering y Data Analysis.

Data Engineer

Infraestructura
Pipelines
Python/Scala

Analytics Engineer

Modelado
Transformación
SQL + dbt

Data Analyst

Dashboards
Reportes
BI Tools

dbt Core vs dbt Cloud

dbt Core - Open Source

- Línea de comandos (CLI)
- Gratuito y open source
- Instalación local
- Requiere orquestación externa
- En este curso

```
pip install dbt-duckdb
```

dbt Cloud - SaaS

- Interfaz web
- IDE en el navegador
- Scheduling integrado
- CI/CD automático
- Plan gratuito para desarrolladores

cloud.getdbt.com

Instalación y Configuración

Instalación de dbt-duckdb

Requisitos previos

- Python 3.9 o superior
- pip (gestor de paquetes)
- Cuenta en MotherDuck con Access Token

Instalación de dbt-duckdb

Instalación

```python
# Crear entorno virtual (recomendado)
python -m venv dbt-env
source dbt-env/bin/activate  # Mac/Linux
# dbt-env\Scripts\activate   # Windows

# Instalar dbt con adaptador DuckDB
pip install dbt-duckdb

# Verificar instalación
dbt --version
```

Inicializar Proyecto dbt

Crear nuevo proyecto

```
# Inicializar proyecto
dbt init mi_proyecto_dbt

# Navegar al proyecto
cd mi_proyecto_dbt
```

Inicializar Proyecto dbt

Preguntas del wizard

- **Which database?** → duckdb
- **path** → md:airbyte_curso (tu base de datos en MotherDuck)

Estructura del Proyecto dbt

Archivos y carpetas principales

```
mi_proyecto_dbt/
├── dbt_project.yml      # Configuración principal del proyecto
├── profiles.yml         # Conexión a la base de datos (en ~/.dbt/)
├── models/              # Aquí van tus modelos SQL
│   ├── staging/         # Modelos de staging (limpieza inicial)
│   ├── intermediate/    # Transformaciones intermedias
│   └── marts/           # Modelos finales para consumo
├── seeds/               # Archivos CSV estáticos
├── macros/              # Funciones reutilizables (Jinja)
├── snapshots/           # Para SCD Tipo 2
└── tests/               # Tests personalizados
```

Configurar profiles.yml

Ubicación: /.dbt/profiles.yml

```yaml
mi_proyecto_dbt:
  outputs:
    dev:
      type: duckdb
      path: "md:airbyte_curso"
      extensions:
        - motherduck
      motherduck_token: "{{ env_var('MOTHERDUCK_TOKEN') }}"
  target: dev
```

Configurar profiles.yml

Configurar token como variable de entorno

```
# Mac/Linux - agregar a ~/.bashrc o ~/.zshrc
export MOTHERDUCK_TOKEN="tu_token_aqui"

# Windows - en PowerShell
$env:MOTHERDUCK_TOKEN="tu_token_aqui"
```

Verificar Conexión

Probar la conexión

```
# Verificar que dbt puede conectarse
dbt debug
```

Verificar Conexión

Output esperado

```
Connection test: [OK connection ok]

All checks passed!
```

Si hay errores, verificar el token y la ruta de la base de datos

☕ Break

15 minutos

Regresamos para escribir nuestros primeros modelos

Modelos en dbt

¿Qué es un Modelo?

Definición

Un **modelo** en dbt es simplemente un archivo `.sql` con una sentencia SELECT que define una transformación.

¿Qué es un Modelo?

Archivo: stg_pokemon.sql

```sql
SELECT
    id as pokemon_id,
    name as pokemon_name,
    height,
    weight,
    base_experience
FROM {{ source('raw', 'pokemon') }}
```

Lo que dbt genera

```sql
CREATE VIEW stg_pokemon AS
SELECT
    id as pokemon_id,
    name as pokemon_name,
    height,
    weight,
    base_experience
FROM raw.pokemon
```

Capas de Modelado

Staging (stg_) - Limpieza y renombrado de columnas

↓

Intermediate (int_) - Joins, agregaciones parciales, lógica de negocio

↓

Marts (dim_, fct_, obt_) - Modelos finales para consumo

Esta estructura facilita mantenimiento, testing y reutilización

Ejemplo: Capa Staging

models/staging/stg_pokemon.sql

```sql
-- Modelo de staging: limpieza básica de datos crudos

WITH source AS (
    SELECT * FROM {{ source('raw', 'pokemon') }}
),

renamed AS (
    SELECT
        id AS pokemon_id,
        name AS pokemon_name,
        height,
        weight,
        base_experience,
        types,
        _airbyte_extracted_at AS loaded_at
    FROM source
)

SELECT * FROM renamed
```

Ejemplo: Capa Intermediate

models/intermediate/int_pokemon_with_types.sql

```sql
-- Extraer tipos del JSON y unir con datos base
WITH pokemon AS (
    SELECT * FROM {{ ref('stg_pokemon') }}
),

pokemon_types AS (
    SELECT
        pokemon_id,
        pokemon_name,
        height,
        weight,
        base_experience,
        types->0->'type'->>'name' AS type_primary,
        types->1->'type'->>'name' AS type_secondary
    FROM pokemon
)

SELECT * FROM pokemon_types
```

Ejemplo: Capa Marts (OBT)

models/marts/obt_pokemon.sql

```sql
-- One Big Table final para análisis
WITH pokemon AS (
    SELECT * FROM {{ ref('int_pokemon_with_types') }}
),
final AS (
    SELECT pokemon_id,
        pokemon_name,
        height,
        weight,
        base_experience,
        type_primary,
        type_secondary,
        CASE
            WHEN base_experience >= 200 THEN 'Legendary'
            WHEN base_experience >= 100 THEN 'Strong'
            ELSE 'Normal'
        END AS power_tier
    FROM pokemon
)
SELECT * FROM final
```

La Función ref()

¿Qué hace ref()?

`{{ ref('nombre_modelo') }}` crea una **referencia** a otro modelo, permitiendo que dbt:

- Construya el **DAG** (grafo de dependencias)
- Ejecute modelos en el **orden correcto**
- Resuelva nombres de esquema automáticamente

La Función ref()

Correcto ✓

```
SELECT *
FROM
{{ ref('stg_pokemon') }}
```

Incorrecto ×

```
SELECT *
FROM main.stg_pokemon
```

La Función source()

¿Qué hace source()?

`{{ source('schema', 'tabla') }}` referencia tablas **externas** a dbt (datos cargados por Airbyte).

La Función source()

Configurar en models/staging/_sources.yml

```yaml
version: 2

sources:
  - name: raw
    database: airbyte_curso
    schema: main
    tables:
      - name: pokemon
      - name: weather
```

Esto permite que dbt sepa de dónde vienen los datos crudos

Materializations

Tipos de Materialización

Tipo	Descripción	Uso típico
view	Crea una VIEW en la DB	Staging, desarrollo
table	Crea una TABLE física	Marts, modelos pesados
incremental	Solo procesa datos nuevos	Fact tables grandes
ephemeral	CTE, no crea objeto en DB	Transformaciones auxiliares

Por defecto: dbt usa `view`. Cambiar a `table` para modelos que se consultan frecuentemente.

Configurar Materialización

En el modelo (config block)

```
{{
  config(
    materialized='table'
  )
}}

SELECT *
FROM {{ ref('stg_pokemon') }}
```

En dbt_project.yml

```yaml
models:
  mi_proyecto:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

Configurar Materialización

La configuración en el modelo tiene prioridad sobre dbt_project.yml

¿Cuándo usar incremental?

Cuando tienes tablas muy grandes y solo necesitas procesar datos **nuevos**.

Ejemplo

```
{{
  config(
    materialized='incremental',
    unique_key='event_id'
  )
}}

SELECT * FROM {{ source('raw', 'events') }}

{% if is_incremental() %}
  WHERE _airbyte_extracted_at > (SELECT MAX(_airbyte_extracted_at) FROM {{ this }})
{% endif %}
```

Ejecutar dbt

Comandos Principales

Comando	Descripción
`dbt run`	Ejecuta todos los modelos
`dbt run --select modelo`	Ejecuta un modelo específico
`dbt run --select staging.*`	Ejecuta todos los modelos en staging
`dbt run --select +modelo`	Ejecuta modelo y sus dependencias upstream
`dbt run --select modelo+`	Ejecuta modelo y sus dependientes downstream
`dbt compile`	Compila SQL sin ejecutar (para debug)
`dbt build`	Run + test en orden correcto

Ver el DAG de Dependencias

Generar documentación con DAG

```
# Generar documentación
dbt docs generate

# Servir localmente (abre navegador)
dbt docs serve
```

Ver el DAG de Dependencias

El DAG muestra

- Todas las **dependencias** entre modelos
- El **orden de ejecución**
- Sources y sus consumidores
- Tests asociados a cada modelo

Debugging: Ver SQL Compilado

¿Dónde está el SQL generado?

Después de ejecutar `dbt run` o `dbt compile`, el SQL compilado está en:

```
target/compiled/mi_proyecto/models/marts/obt_pokemon.sql
```

Debugging: Ver SQL Compilado

Ejemplo de SQL compilado

```sql
-- Las funciones ref() y source() se reemplazan por nombres reales
SELECT *
FROM airbyte_curso.main.stg_pokemon  -- ref() resuelto
```

Actividad Asincrónica

Tarea Práctica - Clase 5

Objetivos

Crear un proyecto dbt completo con modelos en las 3 capas para los datos cargados con Airbyte.

Tarea Práctica - Clase 5

Entregable

Repositorio Git (GitHub/GitLab) con:

1. Proyecto dbt inicializado y configurado
2. Al menos **2 modelos staging** (uno por cada source)
3. Al menos **1 modelo intermediate**
4. Al menos **1 modelo mart** (dimensional u OBT)
5. Archivo `_sources.yml` configurado
6. Captura del **DAG** generado por dbt docs

Estructura Esperada del Proyecto

Organización de archivos

```
mi_proyecto_dbt/
├── dbt_project.yml
├── models/
│   ├── staging/
│   │   ├── _sources.yml
│   │   ├── stg_pokeapi__pokemon.sql
│   │   └── stg_openweather__current.sql
│   ├── intermediate/
│   │   └── int_pokemon_enriched.sql
│   └── marts/
│       └── obt_pokemon_analysis.sql
└── README.md
```

Foro de Discusión - Clase 5

Consigna: Decisiones de diseño en dbt

Comparte las **decisiones de diseño** que tomaste en tu proyecto dbt:

1. ¿Qué **materialización** elegiste para cada capa y por qué?
2. ¿Cómo manejaste los **datos JSON anidados** de las APIs?
3. ¿Optaste por modelo **dimensional** u **OBT**? Justifica.
4. ¿Qué **transformaciones de negocio** agregaste?

Comenta al menos una publicación de un compañero sugiriendo mejoras o alternativas.

Foro de Discusión - Clase 5

⏰ *Participar antes de la próxima clase*

¿Preguntas?

📧 Contacto del docente

📚 Material en plataforma

💬 Foro de consultas abierto

Resumen Clase 5

Conceptos clave

- dbt: transformación con SQL modular
- Capas: staging → intermediate → marts
- ref() y source() para dependencias
- Materializations: view, table, incremental
- DAG de dependencias

Práctica

- Instalar dbt-duckdb
- Configurar profiles.yml con MotherDuck
- Crear modelos SQL por capas
- Ejecutar con dbt run
- Generar docs y visualizar DAG

Resumen Clase 5

Próxima clase: Calidad y Testing de Datos

Tests nativos de dbt y dbt-expectations